

More Ruby on Rails

Action Controller Overview

In this guide, you will learn how controllers work and how they fit into the request cycle in your application.

After reading this guide, you will know how to:

- Follow the flow of a request through a controller.
- Access parameters passed to your controller.
- Use Strong Parameters and permit values.
- Store data in the cookie, the session, and the flash.
- Work with action callbacks to execute code during request processing.
- Use the Request and Response Objects.

1. Introduction

Action Controller is the C in the Model View Controller (**MVC** (<https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93controller>)) pattern. After the **router** ([routing.html](#)) has matched a controller to an incoming request, the controller is responsible for processing the request and generating the appropriate output.

For most conventional **RESTful** (https://en.wikipedia.org/wiki/Representational_state_transfer) applications, the controller will receive the request, fetch or save data from a model, and use a view to create HTML output.

You can imagine that a controller sits between models and views. The controller makes model data available to the view, so that the view can display that data to the user. The controller also receives user input from the view and saves or updates model data accordingly.

2. Creating a Controller

A controller is a Ruby class which inherits from `ApplicationController` and has methods just like any other class. Once an incoming request is matched to a controller by the router, Rails creates an instance of that controller class and calls the method with the same name as the action.

```
class ClientsController < ApplicationController
  def new
    end
end
```

Given the above `ClientsController`, if a user goes to `/clients/new` in your application to add a new client, Rails will create an instance of `ClientsController` and call its `new` method. If the `new` method is empty, Rails will automatically render the `new.html.erb` view by default.



The `new` method is an instance method here, called on an instance of `ClientsController`. This should not be confused with the `new` class method (i.e., `ClientsController.new`).

In the `new` method, the controller would typically create an instance of the `Client` model, and make it available as an instance variable called `@client` in the view:

```
def new
  @client = Client.new
end
```



All controllers inherit from `ApplicationController`, which in turn inherits from `ActionController::Base` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Base.html>). For **API only** (https://guides.rubyonrails.org/api_app.html), applications `ApplicationController` inherits from `ActionController::API` (<https://edgeapi.rubyonrails.org/classes/ActionController/API.html>).

2.1. Controller Naming Convention

Rails favors making the resource in the controller's name plural. For example, `ClientsController` is preferred over `ClientController` and `SiteAdminsController` over `SiteAdminController` or `SitesAdminsController`. However, the plural names are not strictly required (e.g. `ApplicationController`).

Following this naming convention will allow you to use the **default route generators** ([routing.html#crud-verbs-and-actions](#)) (e.g. `resources`) without needing to qualify each with options such as `:controller` ([routing.html#specifying-a-controller-to-use](#)). The convention also makes named route helpers consistent throughout your application.

The controller naming convention is different from models. While plural names are preferred for controller names, the singular form is preferred for **model names** ([active_record_basics.html#naming-conventions](#)) (e.g. `Account` vs. `Accounts`).

Controller actions should be `public`, as only `public` methods are callable as actions. It is also best practice to lower the visibility of helper methods (with `private` or `protected`) which are *not* intended to be actions.



Some method names are reserved by Action Controller. Accidentally redefining them could result in `SystemStackError`. If you limit your controllers to only RESTful **Resource Routing** ([routing.html#resource-routing-the-rails-default](#)), actions you should not need to worry about this.



If you must use a reserved method as an action name, one workaround is to use a custom route to map the reserved method name to your non-reserved action method.

3. Parameters

Data sent by the incoming request is available in your controller in the `params` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/StrongParameters.html#method-i-params>) hash. There are two types of parameter data:

- Query string parameters which are sent as part of the URL (for example, after the `?` in `http://example.com/accounts?filter=free`).
- POST parameters which are submitted from an HTML form.

Rails does not make a distinction between query string parameters and POST parameters; both are available in the `params` hash in your controller. For example:

```
class ClientsController < ApplicationController
  # This action receives query string parameters from an HTTP GET request
  # at the URL "/clients?status=activated"
  def index
    if params[:status] == "activated"
      @clients = Client.activated
    else
      @clients = Client.inactivated
    end
  end

  # This action receives parameters from a POST request to "/clients" URL with form data in
  # the request body.
  def create
    @client = Client.new(params[:client])
    if @client.save
      redirect_to @client
    else
      render "new"
    end
  end
end
```



The `params` hash is not a plain old Ruby Hash; instead, it is an `ActionController::Parameters` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Parameters.html>) object. It does not inherit from Hash, but it behaves mostly like Hash. It provides methods for filtering `params` and, unlike a Hash, keys `:foo` and `"foo"` are considered to be the same.

3.1. Hash and Array Parameters

The `params` hash is not limited to one-dimensional keys and values. It can contain nested arrays and hashes. To send an array of values, append an empty pair of square brackets `[]` to the key name:

```
GET /users?ids[]=1&ids[]=2&ids[]=3
```



The actual URL in this example will be encoded as `/users?ids%5b%5d=1&ids%5b%5d=2&ids%5b%5d=3` as the `[` and `]` characters are not allowed in URLs. Most of the time you don't have to worry about this because the browser will encode it for you, and Rails will decode it automatically, but if you ever find yourself having to send those requests to the server manually you should keep this in mind.

The value of `params[:ids]` will be the array `["1", "2", "3"]`. Note that parameter values are always strings; Rails does not attempt to guess or cast the type.



Values such as `[nil]` or `[nil, nil, ...]` in `params` are replaced with `[]` for security reasons by default. See [Security Guide \(security.html#unsafe-query-generation\)](#) for more information.

To send a hash, you include the key name inside the brackets:

```
<form accept-charset="UTF-8" action="/users" method="post">
  <input type="text" name="user[name]" value="Acme" />
  <input type="text" name="user[phone]" value="12345" />
  <input type="text" name="user[address][postcode]" value="12345" />
  <input type="text" name="user[address][city]" value="Carrot City" />
</form>
```

When this form is submitted, the value of `params[:user]` will be `{ "name" => "Acme", "phone" => "12345", "address" => { "postcode" => "12345", "city" => "Carrot City" } }`. Note the nested hash in `params[:user][:address]`.

The `params` object acts like a Hash, but lets you use symbols and strings interchangeably as keys.

3.2. Composite Key Parameters

Composite key parameters ([active_record_composite_primary_keys.html](#)) contain multiple values in one parameter separated by a delimiter (e.g., an underscore). Therefore, you will need to extract each value so that you can pass them to Active Record. You can use the `extract_value` method to do that.

For example, given the following controller:

```
class BooksController < ApplicationController
  def show
    # Extract the composite ID value from URL parameters.
    id = params.extract_value(:id)
    @book = Book.find(id)
  end
end
```

And this route:

```
get "/books/:id", to: "books#show"
```

When a user requests the URL `/books/4_2`, the controller will extract the composite key value `["4", "2"]` and pass it to `Book.find`. The `extract_value` method may be used to extract arrays out of any delimited parameters.

3.3. JSON Parameters

If your application exposes an API, you will likely accept parameters in JSON format. If the `content-type` header of your request is set to `application/json`, Rails will automatically load your parameters into the `params` hash, which you can access as you would normally.

So for example, if you are sending this JSON content:

```
{ "user": { "name": "acme", "address": "123 Carrot Street" } }
```

Your controller will receive:

```
{ "user" => { "name" => "acme", "address" => "123 Carrot Street" } }
```

3.3.1. Configuring Wrap Parameters

You can use **Wrap Parameters**

(https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/ParamsWrapper/Options/ClassMethods.html#method-i-wrap_parameters), which automatically add the controller name to JSON parameters. For example, you can send the below JSON without a root `:user` key prefix:

```
{ "name": "acme", "address": "123 Carrot Street" }
```

Assuming that you're sending the above data to the `UserController`, the JSON will be wrapped within the `:user` key like this:

```
{ name: "acme", address: "123 Carrot Street", user: { name: "acme", address: "123 Carrot Street" } }
```



Wrap Parameters adds a clone of the parameters to the hash within a key that is the same as the controller name. As a result, both the original version of the parameters and the "wrapped" version of the parameters will exist in the params hash.

This feature clones and wraps parameters with a key chosen based on your controller's name. It is configured to `true` by default. If you do not want to wrap parameters you can [configure \(configuring.html#config-action-controller-wrap-parameters-by-default\)](#) it to `false`.

```
config.action_controller.wrap_parameters_by_default = false
```

You can also customize the name of the key or specific parameters you want to wrap, see the [API documentation \(https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/ParamsWrapper.html\)](#) for more.

3.4. Routing Parameters

Parameters specified as part of a route declaration in the `routes.rb` file are also made available in the `params` hash. For example, we can add a route that captures the `:status` parameter for a client:

```
get "/clients/:status", to: "clients#index", foo: "bar"
```

When a user navigates to `/clients/active` URL, `params[:status]` will be set to `"active"`. When this route is used, `params[:foo]` will also be set to `"bar"`, as if it were passed in the query string.

Any other parameters defined by the route declaration, such as `:id`, will also be available.



In the above example, your controller will also receive `params[:action]` as "index" and `params[:controller]` as "clients". The `params` hash will always contain the `:controller` and `:action` keys, but it's recommended to use the methods `controller_name` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Metal.html#method-i-controller_name) and `action_name` (https://api.rubyonrails.org/v8.1.3.1/classes/AbstractController/Base.html#method-i-action_name) instead to access these values.

3.5. The `default_url_options` Method

You can set global default parameters for `url_for` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionView/RoutingUrlFor.html#method-i-url_for) by defining a method called `default_url_options` in your controller. For example:

```
class ApplicationController < ActionController::Base
  def default_url_options
    { locale: I18n.locale }
  end
end
```

The specified defaults will be used as a starting point when generating URLs. They can be overridden by the options passed to `url_for` or any path helper such as `posts_path`. For example, by setting `locale: I18n.locale`, Rails will automatically add the locale to every URL:

```
posts_path # => "/posts?locale=en"
```

You can still override this default if needed:

```
posts_path(locale: :fr) # => "/posts?locale=fr"
```



Under the hood, `posts_path` is a shorthand for calling `url_for` with the appropriate parameters.

If you define `default_url_options` in `ApplicationController`, as in the example above, these defaults will be used for all URL generation. The method can also be defined in a specific controller, in which case it only applies to URLs generated for that controller.

In a given request, the method is not actually called for every single generated URL. For performance reasons the returned hash is cached per request.

4. Strong Parameters

With Action Controller **Strong Parameters**

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/StrongParameters.html>), parameters cannot be used in Active Model mass assignments until they have been explicitly permitted. This means you will need to decide which attributes to permit for mass update and declare them in the controller. This is a security practice to prevent users from accidentally updating sensitive model attributes.

In addition, parameters can be marked as required and the request will result in a 400 Bad Request being returned if not all required parameters are passed in.

```
class PeopleController < ActionController::Base
  # This will raise an ActiveRecord::ForbiddenAttributesError
  # because it's using mass assignment without an explicit permit.
  def create
    Person.create(params[:person])
  end

  # This will work as we are using `person_params` helper method, which has the
  # call to `expect` to allow mass assignment.
  def update
    person = Person.find(params[:id])
    person.update!(person_params)
    redirect_to person
  end

  private
  # Using a private method to encapsulate the permitted parameters is a good
  # pattern. You can use the same list for both create and update.
  def person_params
    params.expect(person: [:name, :age])
  end
end
```

4.1. Permitting Values

4.1.1. expect

The `expect` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Parameters.html#method-i-expect>) method provides a concise and safe way to require and permit parameters.

```
id = params.expect(:id)
```


The above `expect` will always return a scalar value and not an array or hash. Another example is form params, you can use `expect` to ensure that the root key is present and the attributes are permitted.

```
user_params = params.expect(user: [:username, :password])
user_params.has_key?(:username) # => true
```

In the above example, if the `:user` key is not a nested hash with the specified keys, `expect` will raise an error and return a 400 Bad Request response.

To require and permit an entire hash of parameters, `expect` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Parameters.html#method-i-expect>) can be used in this way.

```
params.expect(log_entry: {})
```

This marks the `:log_entry` parameters hash and any sub-hash of it as permitted and does not check for permitted scalars, anything is accepted.



Extreme care should be taken when calling `expect` with an empty hash, as it will allow all current and future model attributes to be mass-assigned.

4.1.2. permit

Calling `permit` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Parameters.html#method-i-permit>) allows the specified key in `params` (`:id` or `:admin` below) for inclusion in mass assignment (e.g. via `create` or `update`):

```
params = ActionController::Parameters.new(id: 1, admin: "true")
=> #<ActionController::Parameters {"id"=>1, "admin"=>"true"} permitted: false>
params.permit(:id)
=> #<ActionController::Parameters {"id"=>1} permitted: true>
params.permit(:id, :admin)
=> #<ActionController::Parameters {"id"=>1, "admin"=>"true"} permitted: true>
```

For the permitted key `:id`, its value also needs to be one of these permitted scalar values: `String`, `Symbol`, `NilClass`, `Numeric`, `TrueClass`, `FalseClass`, `Date`, `Time`, `DateTime`, `StringIO`, `IO`, `ActionDispatch::Http::UploadedFile`, and `Rack::Test::UploadedFile`.

If you have not called `permit` on the key, it will be filtered out. Arrays, hashes, or any other objects are not injected by default.

To include a value in `params` that's an array of one of the permitted scalar values, you can map the key to an empty array like this:

```
params = ActionController::Parameters.new(tags: ["rails", "parameters"])
=> #<ActionController::Parameters {"tags"=>["rails", "parameters"]} permitted: false>
params.permit(tags: [])
=> #<ActionController::Parameters {"tags"=>["rails", "parameters"]} permitted: true>
```

To include hash values, you can map to an empty hash:

```
params = ActionController::Parameters.new(options: { darkmode: true })
=> #<ActionController::Parameters {"options"=>{"darkmode"=>true}} permitted: false>
params.permit(options: {})
=> #<ActionController::Parameters {"options"=>#<ActionController::Parameters {"darkmode"=>true} permitted: true>} permitted: true>
```

The above `permit` call ensures that values in `options` are permitted scalars and filters out anything else.



The `permit` with an empty hash is convenient since sometimes it is not possible or convenient to declare each valid key of a hash parameter or its internal structure. But note that the above `permit` with an empty hash opens the door to arbitrary input.

4.1.3. `permit!`

There is also `permit!` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Parameters.html#method-i-permit-21>) (with an `!`) which permits an entire hash of parameters without checking the values.

```
params = ActionController::Parameters.new(id: 1, admin: "true")
=> #<ActionController::Parameters {"id"=>1, "admin"=>"true"} permitted: false>
params.permit!
=> #<ActionController::Parameters {"id"=>1, "admin"=>"true"} permitted: true>
```



Extreme care should be taken when using `permit!`, as it will allow all current and *future* model attributes to be mass-assigned.

4.2. Nested Parameters

You can also use `expect` (or `permit`) on nested parameters, like:

```
# Given the example expected params:
params = ActionController::Parameters.new(
  name: "Martin",
  emails: ["me@example.com"],
  friends: [
    { name: "André", family: { name: "RubyGems" }, hobbies: ["keyboards", "card games"] },
    { name: "Kewe", family: { name: "Baroness" }, hobbies: ["video games"] },
  ]
)
# the following expect will ensure the params are permitted
name, emails, friends = params.expect(
  :name,           # permitted scalar
  emails: [],      # array of permitted scalars
  friends: [[      # array of permitted Parameter hashes
    :name,         # permitted scalar
    family: [:name], # family: { name: "permitted scalar" }
    hobbies: []     # array of permitted scalars
  ]]
)
```

This declaration permits the `name`, `emails`, and `friends` attributes and returns them each. It is expected that `emails` will be an array of permitted scalar values, and that `friends` will be an array of resources (note the new double array syntax to explicitly require an array) with specific attributes. These attributes should have a `name` attribute (any permitted scalar values allowed), a `hobbies` attribute as an array of permitted scalar values, and a `family` attribute which is restricted to a hash with only a `name` key and any permitted scalar value.

4.3. Examples

Here are some examples of how to use `permit` for different use cases.

Example 1: You may want to use the permitted attributes in your `new` action. This raises the problem that you can't use `require` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Parameters.html#method-i-require>) on the root key because, normally, it does not exist when calling `new`:

```
# using `fetch` you can supply a default and use
# the Strong Parameters API from there.
params.fetch(:blog, {}).permit(:title, :author)
```

Example 2: The model class method `accepts_nested_attributes_for` allows you to update and destroy associated records. This is based on the `id` and `_destroy` parameters:

```
# permit :id and :_destroy
params.expect(author: [ :name, books_attributes: [ :title, :id, :_destroy ] ])
```

Example 3: Hashes with integer keys are treated differently, and you can declare the attributes as if they were direct children. You get these kinds of parameters when you use `accepts_nested_attributes_for` in combination with a `has_many` association:

```
# To permit the following data:
# {"book" => {"title" => "Some Book",
#           "chapters_attributes" => { "1" => {"title" => "First Chapter"},
#           "2" => {"title" => "Second Chapter"}}}}

params.expect(book: [ :title, chapters_attributes: [[ :title ]] ])
```

Example 4: Imagine a scenario where you have parameters representing a product name, and a hash of arbitrary data associated with that product, and you want to permit the product name attribute and also the whole data hash:

```
def product_params
  params.expect(product: [ :name, data: {} ])
end
```

5. Cookies

The concept of a cookie is not specific to Rails. A [cookie \(https://en.wikipedia.org/wiki/HTTP_cookie\)](https://en.wikipedia.org/wiki/HTTP_cookie) (also known as an HTTP cookie or a web cookie) is a small piece of data from the server that is saved in the user's browser. The browser may store cookies, create new cookies, modify existing ones, and send them back to the server with later requests. Cookies persist data across web requests and therefore enable web applications to remember user preferences.

Rails provides an easy way to access cookies via the [cookies](https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Cookies.html#method-i-cookies) (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Cookies.html#method-i-cookies>) method, which works like a hash:

```
class CommentsController < ApplicationController
  def new
    # Auto-fill the commenter's name if it has been stored in a cookie
    @comment = Comment.new(author: cookies[:commenter_name])
  end

  def create
    @comment = Comment.new(comment_params)
    if @comment.save
      if params[:remember_name]
        # Save the commenter's name in a cookie.
        cookies[:commenter_name] = @comment.author
      else
        # Delete cookie for the commenter's name, if any.
        cookies.delete(:commenter_name)
      end
      redirect_to @comment.article
    else
      render action: "new"
    end
  end
end
```



To delete a cookie, you need to use `cookies.delete(:key)`. Setting the `key` to a `nil` value does not delete the cookie.

When passed a scalar value, the cookie will be deleted when the user closes their browser. If you want the cookie to expire at a specific time, pass a hash with the `:expires` option when setting the cookie. For example, to set a cookie that expires in 1 hour:

```
cookies[:login] = { value: "XJ-122", expires: 1.hour }
```

If you want to create cookies that never expire use the permanent cookie jar. This sets the assigned cookies to have an expiration date 20 years from now.

```
cookies.permanent[:locale] = "fr"
```

5.1. Encrypted and Signed Cookies

Since cookies are stored on the client browser, they can be susceptible to tampering and are not considered secure for storing sensitive data. Rails provides a signed cookie jar and an encrypted cookie jar for storing sensitive data. The signed cookie jar appends a cryptographic signature on the cookie values to protect their integrity. The

encrypted cookie jar encrypts the values in addition to signing them, so that they cannot be read by the user.

Refer to the **API documentation** (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Cookies.html>) for more details.

```
class CookiesController < ApplicationController
  def set_cookie
    cookies.signed[:user_id] = current_user.id
    cookies.encrypted[:expiration_date] = Date.tomorrow # => Thu, 20 Mar 2024
    redirect_to action: "read_cookie"
  end

  def read_cookie
    cookies.encrypted[:expiration_date] # => "2024-03-20"
  end
end
```

These special cookie jars use a serializer to serialize the cookie values into strings and deserialize them into Ruby objects when read back. You can specify which serializer to use via `config.action_dispatch.cookies_serializer` ([configuring.html#config-action-dispatch-cookies-serializer](#)). The default serializer for new applications is `:json`.



Be aware that JSON has limited support serializing Ruby objects such as `Date`, `Time`, and `Symbol`. These will be serialized and deserialized into `Strings`.

If you need to store these or more complex objects, you may need to manually convert their values when reading them in subsequent requests.

If you use the cookie session store, the above applies to the `session` and `flash` hash as well.

6. Session

While cookies are stored client-side, session data is stored server-side (in memory, a database, or a cache), and the duration is usually temporary and tied to the user's session (e.g. until they close the browser). An example use case for session is storing sensitive data like user authentication.

In a Rails application, the session is available in the controller and the view.

6.1. Working with the Session

You can use the `session` instance method to access the session in your controllers. Session values are stored as key/value pairs like a hash:

```
class ApplicationController < ActionController::Base
  private
  # Look up the key `:current_user_id` in the session and use it to
  # find the current `User`. This is a common way to handle user login in
  # a Rails application; logging in sets the session value and
  # logging out removes it.
  def current_user
    @current_user ||= User.find_by(id: session[:current_user_id]) if
session[:current_user_id]
  end
end
```

To store something in the session, you can assign it to a key similar to adding a value to a hash. After a user is authenticated, its `id` is saved in the session to be used for subsequent requests:

```
class SessionsController < ApplicationController
  def create
    if user = User.authenticate_by(email: params[:email], password: params[:password])
      # Save the user ID in the session so it can be used in
      # subsequent requests
      session[:current_user_id] = user.id
      redirect_to root_url
    end
  end
end
```

To remove something from the session, delete the key/value pair. Deleting the `current_user_id` key from the session is a typical way to log the user out:

```
class SessionsController < ApplicationController
  def destroy
    session.delete(:current_user_id)
    # Clear the current user as well.
    @current_user = nil
    redirect_to root_url, status: :see_other
  end
end
```

It is possible to reset the entire session with `reset_session` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Metal.html#method-i-reset_session). It is recommended to use `reset_session` after logging in to avoid session fixation attacks. Please refer to the **Security Guide** (<https://edgeguides.rubyonrails.org/security.html#session-fixation-countermeasures>) for details.



Sessions are lazily loaded. If you don't access sessions in your action's code, they will not be loaded. Hence, you will never need to disable sessions - not accessing them will do the job.

6.2. The Flash

The **flash** (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Flash.html>) provides a way to pass temporary data between controller actions. Anything you place in the flash will be available to the very next action and then cleared. The flash is typically used for setting messages (e.g. notices and alerts) in a controller action before redirecting to an action that displays the message to the user.

The flash is accessed via the `flash` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Flash/RequestMethods.html#method-i-flash>) method. Similar to the session, the flash values are stored as key/value pairs.

For example, in the controller action for logging out a user, the controller can set a flash message which can be displayed to the user on the next request:

```
class SessionsController < ApplicationController
  def destroy
    session.delete(:current_user_id)
    flash[:notice] = "You have successfully logged out."
    redirect_to root_url, status: :see_other
  end
end
```

Displaying a message after a user performs some interactive action in your application is a good practice to give the user feedback that their action was successful (or that there were errors).

In addition to `:notice`, you can also use `:alert`. These are typically styled (using CSS) with different colors to indicate their meaning (e.g. green for notices and orange/red for alerts).

You can also assign a flash message directly within the `redirect_to` method by including it as a parameter to `redirect_to`:

```
redirect_to root_url, notice: "You have successfully logged out."
redirect_to root_url, alert: "There was an issue."
```

You're not limited to `notice` and `alert`. You can set any key in a flash (similar to sessions), by assigning it to the `:flash` argument. For example, assigning `:just_signed_up`:

```
redirect_to root_url, flash: { just_signed_up: true }
```

This will allow you to have the below in the view:


```
<% if flash[:just_signed_up] %>
  <p class="welcome">Welcome to our site!</p>
<% end %>
```

In the above logout example, the `destroy` action redirects to the application's `root_url`, where the message is available to be displayed. However, it's not displayed automatically. It's up to the next action to decide what, if anything, it will do with what the previous action put in the flash.

6.2.1. Displaying flash messages

If a previous action *has* set a flash message, it's a good idea to display that to the user. We can accomplish this consistently by adding the HTML for displaying any flash messages in the application's default layout. Here's an example from `app/views/layouts/application.html.erb`:

```
<html>
  <!-- <head/> -->
  <body>
    <% flash.each do |name, msg| -%>
      <%= content_tag :div, msg, class: name %>
    <% end -%>

    <!-- more content -->
    <%= yield %>
  </body>
</html>
```

The `name` above indicates the type of flash message, such as `notice` or `alert`. This information is normally used to style how the message is displayed to the user.



You can filter by `name` if you want to limit to displaying only `notice` and `alert` in layout. Otherwise, all keys set in the `flash` will be displayed.

Including the reading and displaying of flash messages in the layout ensures that your application will display these automatically, without each view having to include logic to read the flash.

6.2.2. `flash.keep` and `flash.now`

`flash.keep` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Flash/FlashHash.html#method-i-keep>) is used to carry over the flash value through to an additional request. This is useful when there are multiple redirects.

For example, assume that the `index` action in the controller below corresponds to the `root_url`. And you want all requests here to be redirected to `UserController#index`. If an action sets the flash and redirects to `MainController#index`, those flash values will be lost when another redirect happens, unless you use

`flash.keep` to make the values persist for another request.

```
class MainController < ApplicationController
  def index
    # Will persist all flash values.
    flash.keep

    # You can also use a key to keep only some kind of value.
    # flash.keep(:notice)
    redirect_to users_url
  end
end
```

`flash.now` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Flash/FlashHash.html#method-i-now>)

is used to make the flash values available in the same request. By default, adding values to the flash will make them available to the next request. For example, if the `create` action fails to save a resource, and you render the `new` template directly, that's not going to result in a new request, but you may still want to display a message using the flash. To do this, you can use `flash.now`

(<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Flash/FlashHash.html#method-i-now>) in the same way you use the normal `flash`:

```
class ClientsController < ApplicationController
  def create
    @client = Client.new(client_params)
    if @client.save
      # ...
    else
      flash.now[:error] = "Could not save client"
      render action: "new"
    end
  end
end
```

6.3. Session Stores

All sessions have a unique ID that represents the session object; these session IDs are stored in a cookie. The actual session objects use one of the following storage mechanisms:

- `ActionDispatch::Session::CookieStore` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Session/CookieStore.html>) - Stores everything on the client.
- `ActionDispatch::Session::CacheStore` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Session/CacheStore.html>) - Stores the data in the Rails cache.

- `ActionDispatch::Session::ActiveRecordStore` (https://github.com/rails/activerecord-session_store) - Stores the data in a database using Active Record (requires the `activerecord-session_store` (https://github.com/rails/activerecord-session_store) gem).
- A custom store or a store provided by a third party gem.

For most session stores, the unique session ID in the cookie is used to look up session data on the server (e.g. a database table). Rails does not allow you to pass the session ID in the URL as this is less secure.

6.3.1. CookieStore

The `CookieStore` is the default and recommended session store. It stores all session data in the cookie itself (the session ID is still available to you if you need it). The `CookieStore` is lightweight and does not require any configuration to use in a new application.

The `CookieStore` can store 4 kB of data - much less than the other storage options - but this is usually enough. Storing large amounts of data in the session is discouraged. You should especially avoid storing complex objects (such as model instances) in the session.

6.3.2. CacheStore

You can use the `CacheStore` if your sessions don't store critical data or don't need to be around for long periods (for instance if you just use the flash for messaging). This will store sessions using the cache implementation you have configured for your application. The advantage is that you can use your existing cache infrastructure for storing sessions without requiring any additional setup or administration. The downside is that the session storage will be temporary and they could disappear at any time.

Read more about session storage in the [Security Guide \(security.html#sessions\)](#).

6.4. Session Storage Options

There are a few configuration options related to session storage. You can configure the type of storage in an initializer:

```
Rails.application.config.session_store :cache_store
```

Rails sets up a session key (the name of the cookie) when signing the session data. These can also be changed in an initializer:

```
Rails.application.config.session_store :cookie_store, key: "_your_app_session"
```



Be sure to restart your server when you modify an initializer file.

You can also pass a `:domain` key and specify the domain name for the cookie:

```
Rails.application.config.session_store :cookie_store, key: "_your_app_session", domain: ".example.com"
```



See [config.session_store \(configuring.html#config-session-store\)](#) in the configuration guide for more information.

Rails sets up a secret key for `CookieStore` used for signing the session data in `config/credentials.yml.enc`. The credentials can be updated with `bin/rails credentials:edit`.

```
# aws:
#   access_key_id: 123
#   secret_access_key: 345

# Used as the base secret for all MessageVerifiers in Rails, including the one protecting
# cookies.
secret_key_base: 492f...
```



Changing the `secret_key_base` when using the `CookieStore` will invalidate all existing sessions. You'll need to configure a **cookie rotator** (<https://edgeguides.rubyonrails.org/configuring.html#config-action-dispatch-cookies-rotations>) to rotate existing sessions.

7. Controller Callbacks

Controller callbacks are methods that are defined to automatically run before and/or after a controller action. A controller callback method can be defined in a given controller or in the `ApplicationController`. Since all controllers inherit from `ApplicationController`, callbacks defined here will run on every controller in your application.

7.1. `before_action`

Callback methods registered via [before action](#) (https://api.rubyonrails.org/v8.1.3.1/classes/AbstractController/Callbacks/ClassMethods.html#method-i-before_action) run *before* a controller action. They may halt the request cycle. A common use case for `before_action` is ensuring that a user is logged in:

```
class ApplicationController < ActionController::Base
  before_action :require_login

  private
  def require_login
    unless logged_in?
      flash[:error] = "You must be logged in to access this section"
      redirect_to new_login_url # halts request cycle
    end
  end
end
```

The method stores an error message in the flash and redirects to the login form if the user is not already logged in. When a `before_action` callback renders or redirects (like in the example above), the original controller action is not run. If there are additional callbacks registered to run, they are also cancelled and not run.

In this example, the `before_action` is defined in `ApplicationController` so all controllers in the application inherit it. That implies that all requests in the application will require the user to be logged in. This is fine except for the "login" page. The "login" action should succeed even when the user is not logged in (to allow the user to log in) otherwise the user will never be able to log in. You can use `skip_before_action` (https://api.rubyonrails.org/v8.1.3.1/classes/AbstractController/Callbacks/ClassMethods.html#method-i-skip_before_action) to allow specified controller actions to skip a given `before_action`:

```
class LoginsController < ApplicationController
  skip_before_action :require_login, only: [:new, :create]
end
```

Now, the `LoginsController`'s `new` and `create` actions will work without requiring the user to be logged in.

The `:only` option skips the callback only for the listed actions; there is also an `:except` option which works the other way. These options can be used when registering action callbacks too, to add callbacks which only run for selected actions.



If you register the same action callback multiple times with different options, the last action callback definition will overwrite the previous ones.

7.2. `after_action` and `around_action`

You can also define action callbacks to run *after* a controller action has been executed with `after_action` (https://api.rubyonrails.org/v8.1.3.1/classes/AbstractController/Callbacks/ClassMethods.html#method-i-after_action), or to run both before and after with `around_action` (https://api.rubyonrails.org/v8.1.3.1/classes/AbstractController/Callbacks/ClassMethods.html#method-i-around_action).

The `after_action` callbacks are similar to `before_action` callbacks, but because the controller action has already been run they have access to the response data that's about to be sent to the client.



`after_action` callbacks are executed only after a successful controller action, and not if an exception is raised in the request cycle.

The `around_action` callbacks are useful when you want to execute code before and after a controller action, allowing you to encapsulate functionality that affects the action's execution. They are responsible for running their associated actions by yielding.

For example, imagine you want to monitor the performance of specific actions. You could use an `around_action` to measure how long each action takes to complete and log this information:

```
class ApplicationController < ActionController::Base
  around_action :measure_execution_time

  private
  def measure_execution_time
    start_time = Time.now
    yield # This executes the action
    end_time = Time.now

    duration = end_time - start_time
    Rails.logger.info "Action #{action_name} from controller #{controller_name} took #{
      {duration.round(2)} seconds to execute."
    }
  end
end
```



Action callbacks receive `controller_name` and `action_name` as parameters you can use, as shown in the example above.

The `around_action` callback also wraps rendering. In the example above, view rendering will be included in the `duration`. The code after the `yield` in an `around_action` is run even when there is an exception in the associated action and there is an `ensure` block in the callback. (This is different from `after_action` callbacks where an exception in the action cancels the `after_action` code.)

7.3. Other Ways to Use Callbacks

In addition to `before_action`, `after_action`, or `around_action`, there are two less common ways to register callbacks.

The first is to use a block directly with the `*_action` methods. The block receives the controller as an argument. For example, the `require_login` action callback from above could be rewritten to use a block:

```
class ApplicationController < ActionController::Base
  before_action do |controller|
    unless controller.send(:logged_in?)
      flash[:error] = "You must be logged in to access this section"
      redirect_to new_login_url
    end
  end
end
```

Note that the action callback, in this case, uses `send` because the `logged_in?` method is private, and the action callback does not run in the scope of the controller. This is not the recommended way to implement this particular action callback, but in simpler cases, it might be useful.

Specifically for `around_action`, the block also yields in the `action`:

```
around_action { |_controller, action| time(&action) }
```

The second way is to specify a class (or any object that responds to the expected methods) for the callback action. This can be useful in cases that are more complex. As an example, you could rewrite the `around_action` callback to measure execution time with a class:

```
class ApplicationController < ActionController::Base
  around_action ActionDurationCallback
end

class ActionDurationCallback
  def self.around(controller)
    start_time = Time.now
    yield # This executes the action
    end_time = Time.now

    duration = end_time - start_time
    Rails.logger.info "Action #{controller.action_name} from controller #{controller.controller_name} took #{duration.round(2)} seconds to execute."
  end
end
```

In the above example, the `ActionDurationCallback`'s method is not run in the scope of the controller but gets `controller` as an argument.

In general, the class being used for a `*_action` callback must implement a method with the same name as the action callback. So for the `before_action` callback, the class must implement a `before` method, and so on. Also, the `around` method must `yield` to execute the action.

8. The Request and Response Objects

Every controller has two methods, `request` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Base.html#method-i-request>) and `response` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Base.html#method-i-response>), which can be used to access the request and response objects associated with the current request cycle. The `request` method returns an instance of `ActionDispatch::Request` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Request.html>). The `response` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionController/Base.html#method-i-response>) method returns an instance of `ActionDispatch::Response` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Response.html>), an object representing what is going to be sent back to the client browser (e.g. from `render` or `redirect` in the controller action).

8.1. The `request` Object

The request object contains useful information about the request coming in from the client. This section describes the purpose of some of the properties of the `request` object.

To get a full list of the available methods, refer to the [Rails API documentation](https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Request.html) (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Request.html>) and [Rack](https://rack.github.io/rack/main/Rack/Request.html) (<https://rack.github.io/rack/main/Rack/Request.html>) documentation.

Property of <code>request</code>	Purpose
<code>host</code>	The hostname used for this request.
<code>domain(n=2)</code>	The hostname's first n segments, starting from the right (the TLD).
<code>format</code>	The content type requested by the client.
<code>method</code>	The HTTP method used for the request.
<code>get?, post?, patch?, put?, delete?, head?</code>	Returns true if the HTTP method is GET/POST/PATCH/PUT/DELETE/HEAD.
<code>headers</code>	Returns a hash containing the headers associated with the request.
<code>port</code>	The port number (integer) used for the request.
<code>protocol</code>	Returns a string containing the protocol used plus "://", for example "http://".
<code>query_string</code>	The query string part of the URL, i.e., everything after "?".
<code>remote_ip</code>	The IP address of the client.
<code>url</code>	The entire URL used for the request.

8.1.1. `query_parameters`, `request_parameters`, and `path_parameters`

Rails collects all of the parameters for a given request in the `params` hash, including the ones set in the URL as query string parameters, and those sent as the body of a POST request. The request object has three methods that give you access to the various parameters.

- `query_parameters` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Request.html#method-i-query_parameters) - contains parameters that were sent as part of the query string.
- `request_parameters` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Request.html#method-i-request_parameters) - contains parameters sent as part of the post body.
- `path_parameters` (https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Http/Parameters.html#method-i-path_parameters) - contains parameters parsed by the router as being part of the path leading to this particular controller and action.

8.1.2. `request.variant`

Controllers might need to tailor a response based on context-specific information in a request. For example, controllers responding to requests from a mobile platform might need to render different content than requests from a desktop browser. One strategy to accomplish this is by customizing a request's variant. Variant names are arbitrary, and can communicate anything from the request's platform (`:android`, `:ios`, `:linux`, `:macos`, `:windows`) to its browser (`:chrome`, `:edge`, `:firefox`, `:safari`), to the type of user (`:admin`, `:guest`, `:user`).

You can set the `request.variant` (<https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Http/MimeNegotiation.html#method-i-variant-3D>) in a `before_action`:

```
request.variant = :tablet if request.user_agent.include?("iPad")
```

Responding with a variant in a controller action is like responding with a format:

```
# app/controllers/projects_controller.rb

def show
  # ...
  respond_to do |format|
    format.html do |html|
      html.tablet # renders app/views/projects/show.html+tablet.erb
      html.phone { extra_setup; render } # renders app/views/projects/show.html+phone.erb
    end
  end
end
```

A separate template should be created for each format and variant:

- `app/views/projects/show.html.erb`
- `app/views/projects/show.html+tablet.erb`
- `app/views/projects/show.html+phone.erb`

You can also simplify the variants definition using the inline syntax:

```
respond_to do |format|
  format.html.tablet
  format.html.phone { extra_setup; render }
end
```

8.2. The `response` Object

The response object is built up during the execution of the action from rendering data to be sent back to the client browser. It's not usually used directly but sometimes, in an `after_action` callback for example, it can be useful to access the response directly. One use case is for setting the content type header:

```
response.content_type = "application/pdf"
```

Another use case is for setting custom response headers:

```
response.headers["X-Custom-Header"] = "some value"
```

The `headers` attribute is a hash which maps header names to header values. Rails sets some headers automatically but if you need to update a header or add a custom header, you can use `response.headers` as in the example above.



The `headers` method can be accessed directly in the controller as well.

Here are some of the properties of the `response` object:

Property of <code>response</code>	Purpose
<code>body</code>	This is the string of data being sent back to the client. This is most often HTML.
<code>status</code>	The HTTP status code for the response, like 200 for a successful request or 404 for file not found.

Property of response	Purpose
location	The URL the client is being redirected to, if any.
content_type	The content type of the response.
charset	The character set being used for the response. Default is "utf-8".
headers	Headers used for the response.

To get a full list of the available methods, refer to the [Rails API documentation \(https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Response.html\)](https://api.rubyonrails.org/v8.1.3.1/classes/ActionDispatch/Response.html) and [Rack Documentation \(https://rack.github.io/rack/main/Rack/Response.html\)](https://rack.github.io/rack/main/Rack/Response.html).



Feedback

You're encouraged to help improve the quality of this guide.

Please contribute if you see any typos or factual errors. To get started, you can read our [documentation contributions \(https://edgeguides.rubyonrails.org/contributing_to_ruby_on_rails.html#contributing-to-the-rails-documentation\)](https://edgeguides.rubyonrails.org/contributing_to_ruby_on_rails.html#contributing-to-the-rails-documentation) section.

You may also find incomplete content or stuff that is not up to date. Please do add any missing documentation for main. Make sure to check [Edge Guides \(https://edgeguides.rubyonrails.org\)](https://edgeguides.rubyonrails.org) first to verify if the issues are already fixed or not on the main branch. Check the [Ruby on Rails Guides Guidelines \(ruby_on_rails_guides_guidelines.html\)](https://edgeguides.rubyonrails.org/ruby_on_rails_guides_guidelines.html) for style and conventions.

If for whatever reason you spot something to fix but cannot patch it yourself, please [open an issue \(https://github.com/rails/rails/issues\)](https://github.com/rails/rails/issues).

And last but not least, any kind of discussion regarding Ruby on Rails documentation is very welcome on the [official Ruby on Rails Forum \(https://discuss.rubyonrails.org/c/rubyonrails-docs\)](https://discuss.rubyonrails.org/c/rubyonrails-docs).

This work is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International \(https://creativecommons.org/licenses/by-sa/4.0/\)](https://creativecommons.org/licenses/by-sa/4.0/) License

"Rails", "Ruby on Rails", and the Rails logo are trademarks of David Heinemeier Hansson. All rights reserved.